

# **PYTHON PROGRAMMING**

## **OPERATORS**

# Arithmetic Operators in Python

| Operator | Operation      | Description                                                                                                                                                                | Example (Try in Lab)                                                                                                                                                             |
|----------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| +        | Addition       | Adds the two numeric values on either side of the operator<br><br>This operator can also be used to concatenate two strings on either side of the operator                 | <pre>&gt;&gt;&gt; num1 = 5 &gt;&gt;&gt; num2 = 6 &gt;&gt;&gt; num1 + num2 11 &gt;&gt;&gt; str1 = "Hello" &gt;&gt;&gt; str2 = "India" &gt;&gt;&gt; str1 + str2 'HelloIndia'</pre> |
| -        | Subtraction    | Subtracts the operand on the right from the operand on the left                                                                                                            | <pre>&gt;&gt;&gt; num1 = 5 &gt;&gt;&gt; num2 = 6 &gt;&gt;&gt; num1 - num2 -1</pre>                                                                                               |
| *        | Multiplication | Multiplies the two values on both side of the operator<br><br>Repeats the item on left of the operator if first operand is a string and second operand is an integer value | <pre>&gt;&gt;&gt; num1 = 5 &gt;&gt;&gt; num2 = 6 &gt;&gt;&gt; num1 * num2 30 &gt;&gt;&gt; str1 = 'India' &gt;&gt;&gt; str1 * 2 'IndiaIndia'</pre>                                |

|    |                |                                                                                                                                                                  |                                                                                 |
|----|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| /  | Division       | Divides the operand on the left by the operand on the right and returns the quotient                                                                             | >>> num1 = 8<br>>>> num2 = 4<br>>>> num2 / num1<br>0.5                          |
| %  | Modulus        | Divides the operand on the left by the operand on the right and returns the remainder                                                                            | >>> num1 = 13<br>>>> num2 = 5<br>>>> num1 % num2<br>3                           |
| // | Floor Division | Divides the operand on the left by the operand on the right and returns the quotient by removing the decimal part. It is sometimes also called integer division. | >>> num1 = 13<br>>>> num2 = 4<br>>>> num1 // num2<br>3<br>>>> num2 // num1<br>0 |
| ** | Exponent       | Performs exponential (power) calculation on operands. That is, raise the operand on the left to the power of the operand on the right                            | >>> num1 = 3<br>>>> num2 = 4<br>>>> num1 ** num2<br>81                          |

## Relational Operators

Relational operator compares the values of the operands on its either side and determines the relationship among them. Assume the Python variables `num1 = 10`, `num2 = 0`, `num3 = 10`, `str1 = "Good"`, `str2 = "Afternoon"` for the following examples:

## Relational operators in Python

| Operator | Operation    | Description                                                                                                                              | Example (Try in Lab)                                                              |
|----------|--------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| ==       | Equals to    | If the values of two operands are equal, then the condition is True, otherwise it is False                                               | >>> num1 == num2<br>False<br>>> str1 == str2<br>False                             |
| !=       | Not equal to | If values of two operands are not equal, then condition is True, otherwise it is False                                                   | >>> num1 != num2<br>True<br>>>> str1 != str2<br>True<br>>>> num1 != num3<br>False |
| >        | Greater than | If the value of the left-side operand is greater than the value of the right-side operand, then condition is True, otherwise it is False | >>> num1 > num2<br>True<br>>>> str1 > str2<br>True                                |

|    |                          |                                                                                                                                                      |                                                                                                                    |
|----|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <  | Less than                | If the value of the left-side operand is less than the value of the right-side operand, then condition is True, otherwise it is False                | <pre>&gt;&gt;&gt; num1 &lt; num3 False &gt;&gt;&gt; str2 &lt; str1 True</pre>                                      |
| >= | Greater than or equal to | If the value of the left-side operand is greater than or equal to the value of the right-side operand, then condition is True, otherwise it is False | <pre>&gt;&gt;&gt; num1 &gt;= num2 True &gt;&gt;&gt; num2 &gt;= num3 False &gt;&gt;&gt; str1 &gt;= str2 True</pre>  |
| <= | Less than or equal to    | If the value of the left operand is less than or equal to the value of the right operand, then is True otherwise it is False                         | <pre>&gt;&gt;&gt; num1 &lt;= num2 False &gt;&gt;&gt; num2 &lt;= num3 True &gt;&gt;&gt; str1 &lt;= str2 False</pre> |

# Assignment Operators

Assignment operator assigns or changes the value of the variable on its left.

**Table 5.5 Assignment operators in Python**

| Operator | Description                                                                                                                                                  | Example (Try in Lab)                                                                                                                                                                                                                        |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| =        | Assigns value from right-side operand to left-side operand                                                                                                   | <pre>&gt;&gt;&gt; num1 = 2 &gt;&gt;&gt; num2 = num1 &gt;&gt;&gt; num2 2 &gt;&gt;&gt; country = 'India' &gt;&gt;&gt; country 'India'</pre>                                                                                                   |
| +=       | It adds the value of right-side operand to the left-side operand and assigns the result to the left-side operand<br><b>Note:</b> x += y is same as x = x + y | <pre>&gt;&gt;&gt; num1 = 10 &gt;&gt;&gt; num2 = 2 &gt;&gt;&gt; num1 += num2 &gt;&gt;&gt; num1 12 &gt;&gt;&gt; num2 2 &gt;&gt;&gt; str1 = 'Hello' &gt;&gt;&gt; str2 = 'India' &gt;&gt;&gt; str1 += str2 &gt;&gt;&gt; str1 'HelloIndia'</pre> |

|           |                                                                                                                                                                                                              |                                                                                                                                                                                  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>-=</p> | <p>It subtracts the value of right-side operand from the left-side operand and assigns the result to left-side operand<br/> <b>Note:</b> <math>x -= y</math> is same as <math>x = x - y</math></p>           | <pre>&gt;&gt;&gt; num1 = 10 &gt;&gt;&gt; num2 = 2 &gt;&gt;&gt; num1 -= num2 &gt;&gt;&gt; num1 8</pre>                                                                            |
| <p>*=</p> | <p>It multiplies the value of right-side operand with the value of left-side operand and assigns the result to left-side operand<br/> <b>Note:</b> <math>x *= y</math> is same as <math>x = x * y</math></p> | <pre>&gt;&gt;&gt; num1 = 2 &gt;&gt;&gt; num2 = 3 &gt;&gt;&gt; num1 *= 3  &gt;&gt;&gt; num1 6 &gt;&gt;&gt; a = 'India' &gt;&gt;&gt; a *= 3 &gt;&gt;&gt; a 'IndiaIndiaIndia'</pre> |



|                  |                                                                                                                                                                                                           |                                                                                                        |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>/=</code>  | <p>It divides the value of left-side operand by the value of right-side operand and assigns the result to left-side operand</p> <p><b>Note:</b> <code>x /= y</code> is same as <code>x = x / y</code></p> | <pre>&gt;&gt;&gt; num1 = 6 &gt;&gt;&gt; num2 = 3 &gt;&gt;&gt; num1 /= num2 &gt;&gt;&gt; num1 2.0</pre> |
| <code>%=</code>  | <p>It performs modulus operation using two operands and assigns the result to left-side operand</p> <p><b>Note:</b> <code>x %= y</code> is same as <code>x = x % y</code></p>                             | <pre>&gt;&gt;&gt; num1 = 7 &gt;&gt;&gt; num2 = 3 &gt;&gt;&gt; num1 %= num2 &gt;&gt;&gt; num1 1</pre>   |
| <code>//=</code> | <p>It performs floor division using two operands and assigns the result to left-side operand</p> <p><b>Note:</b> <code>x //= y</code> is same as <code>x = x // y</code></p>                              | <pre>&gt;&gt;&gt; num1 = 7 &gt;&gt;&gt; num2 = 3 &gt;&gt;&gt; num1 //= num2 &gt;&gt;&gt; num1 2</pre>  |
| <code>**=</code> | <p>It performs exponential (power) calculation on operators and assigns value to the left-side operand</p> <p><b>Note:</b> <code>x **= y</code> is same as <code>x = x ** y</code></p>                    | <pre>&gt;&gt;&gt; num1 = 2 &gt;&gt;&gt; num2 = 3 &gt;&gt;&gt; num1 **= num2 &gt;&gt;&gt; num1 8</pre>  |

## 5.8.4 Logical Operators

There are three logical operators supported by Python. These operators (`and`, `or`, `not`) are to be written in lower case only. The logical operator evaluates to either `True` or `False` based on the logical operands on either side. Every value is logically either `True` or `False`. By default, all values are `True` except `None`, `False`, `0` (zero), empty collections `""`, `()`, `[]`, `{}`, and few other special values. So if we say `num1 = 10`, `num2 = -20`, then both `num1` and `num2` are logically `True`.

## Logical operators in Python

| Operator | Operation   | Description                                                      | Example (Try in Lab)                                                                                                                                                                                                                                              |
|----------|-------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| and      | Logical AND | If both the operands are True, then condition becomes True       | <pre>&gt;&gt;&gt; True and True True &gt;&gt;&gt; num1 = 10 &gt;&gt;&gt; num2 = -20 &gt;&gt;&gt; bool(num1 and num2) True &gt;&gt;&gt; True and False False &gt;&gt;&gt; num3 = 0 &gt;&gt;&gt; bool(num1 and num3) False &gt;&gt;&gt; False and False False</pre> |
| or       | Logical OR  | If any of the two operands are True, then condition becomes True | <pre>&gt;&gt;&gt; True or True True &gt;&gt;&gt; True or False True &gt;&gt;&gt; bool(num1 or num3) True &gt;&gt;&gt; False or False False</pre>                                                                                                                  |

|     |             |                                                  |                                                                                                                    |
|-----|-------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| not | Logical NOT | Used to reverse the logical state of its operand | <pre>&gt;&gt;&gt; num1 = 10 &gt;&gt;&gt; bool(num1) True &gt;&gt;&gt; not num1 &gt;&gt;&gt; bool(num1) False</pre> |
|-----|-------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|

## Identity Operators

Identity operators are used to determine whether the value of a variable is of a certain type or not. Identity operators can also be used to determine whether two variables are referring to the same object or not. There are two identity operators.

**Table 5.7 Identity operators in Python**

| Operator | Description                                                                                                                                                                                    | Example (Try in Lab)                                                                                                                                     |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| is       | Evaluates True if the variables on either side of the operator point towards the same memory location and False otherwise. var1 is var2 results to True if id(var1) is equal to id(var2)       | >>> num1 = 5<br>>>> type(num1) is int<br>True<br>>>> num2 = num1<br>>>> id(num1)<br>1433920576<br>>>> id(num2)<br>1433920576<br>>>> num1 is num2<br>True |
| is not   | Evaluates to False if the variables on either side of the operator point to the same memory location and True otherwise. var1 is not var2 results to True if id(var1) is not equal to id(var2) | >>> num1 is not num2<br>False                                                                                                                            |

## 5.8.6 Membership Operators

Membership operators are used to check if a value is a member of the given sequence or not.

**Table 5.8 Membership operators in Python**

| Operator | Description                                                                                   | Example (Try in Lab)                                                                            |
|----------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| in       | Returns True if the variable/value is found in the specified sequence and False otherwise     | <pre>&gt;&gt;&gt; a = [1,2,3] &gt;&gt;&gt; 2 in a True &gt;&gt;&gt; '1' in a False</pre>        |
| not in   | Returns True if the variable/value is not found in the specified sequence and False otherwise | <pre>&gt;&gt;&gt; a = [1,2,3] &gt;&gt;&gt; 10 not in a True &gt;&gt;&gt; 1 not in a False</pre> |

## 5.9 EXPRESSIONS

An expression is defined as a combination of constants, variables, and operators. An expression always evaluates to a value. A value or a standalone variable is also considered as an expression but a standalone operator is not an expression. Some examples of valid expressions are given below.

- |                           |                            |
|---------------------------|----------------------------|
| (i) 100                   | (iv) $3.0 + 3.14$          |
| (ii) num                  | (v) $23/3 - 5 * 7(14 - 2)$ |
| (iii) $\text{num} - 20.4$ | (vi) "Global" + "Citizen"  |



### 5.9.1 Precedence of Operators

Evaluation of the expression is based on precedence of operators. When an expression contains different kinds of operators, precedence determines which operator should be applied first. Higher precedence operator is evaluated before the lower precedence operator. Most of the operators studied till now are binary operators. Binary operators are operators with two operands. The unary operators need only one operand, and they have a higher precedence than the binary operators. The minus (-) as well as + (plus) operators can act as both unary and binary operators, but not is a unary logical operator.

```
#Depth is using - (minus) as unary operator
Value = -Depth
#not is a unary operator, negates True
print(not(True))
```

**Table 5.9 Precedence of all operators in Python**

| Order of Precedence | Operators                              | Description                                 |
|---------------------|----------------------------------------|---------------------------------------------|
| 1                   | **                                     | Exponentiation (raise to the power)         |
| 2                   | ~ , +, -                               | Complement, unary plus and unary minus      |
| 3                   | * , / , % , //                         | Multiply, divide, modulo and floor division |
| 4                   | +, -                                   | Addition and subtraction                    |
| 5                   | <= , < , > , >= , == , !=              | Relational and Comparison operators         |
| 6                   | = , %= , /= , //= , -= , += , *= , **= | Assignment operators                        |
| 7                   | is , is not                            | Identity operators                          |
| 8                   | in , not in                            | Membership operators                        |
| 9                   | not                                    | Logical operators                           |
| 10                  | and                                    |                                             |
| 11                  | or                                     |                                             |

**Note:**

- Parenthesis can be used to override the precedence of operators. The expression within () is evaluated first.
- For operators with equal precedence, the expression is evaluated from left to right.

*Example 5.9* How will Python evaluate the following expression?

$$20 + 30 * 40$$

*Solution:*

|                                         |                    |         |
|-----------------------------------------|--------------------|---------|
|                                         | $= 20 + (30 * 40)$ | #Step 1 |
| #precedence of * is more than that of + |                    |         |
|                                         | $= 20 + 1200$      | #Step 2 |
|                                         | $= 1220$           | #Step 3 |

*Example 5.10* How will Python evaluate the following expression?

$$20 - 30 + 40$$

*Solution:*

The two operators (-) and (+) have equal precedence. Thus, the first operator, i.e., subtraction is applied before the second operator, i.e., addition (left to right).

$$= (20 - 30) + 40 \quad \text{\#Step 1}$$

$$= -10 + 40 \quad \text{\#Step 2}$$

$$= 30 \quad \text{\#Step 3}$$

*Example 5.11* How will Python evaluate the following expression?

`(20 + 30) * 40`

*Solution:*

`= (20 + 30) * 40` # Step 1

#using parenthesis(), we have forced precedence  
of + to be more than that of \*

`= 50 * 40` # Step 2

`= 2000` # Step 3

*Example 5.12* How will the following expression be evaluated in Python?

`15.0 / 4 + (8 + 3.0)`

*Solution:*

```
= 15.0 / 4 + (8.0 + 3.0)  #Step 1
= 15.0 / 4.0 + 11.0       #Step 2
= 3.75 + 11.0             #Step 3
= 14.75                   #Step 4
```